本章路線

這章要回答的核心問題是：作業系統如何表示、切換與協調彼此獨立又可能合作的執行單元？

前置知識。 CPU 暫存器、記憶體位址空間、中斷與 system call。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

學習目標

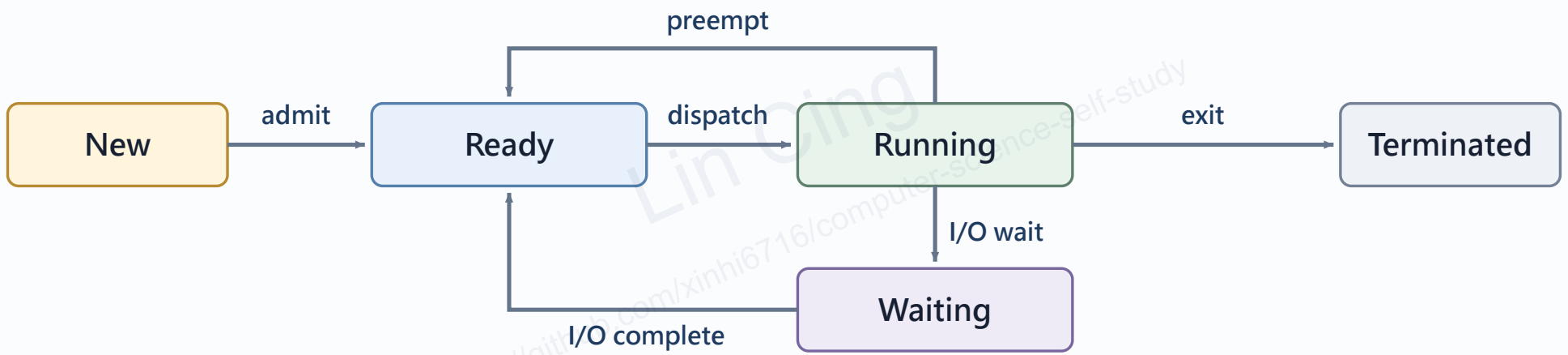
- 追蹤 process state 與 PCB 內容
- 區分排程器、dispatcher 與 context switch 成本
- 設計具明確 blocking、buffering 與終止語意的 IPC

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.1 Process concept

- process 是執行中的程式，包含 text、data、heap、stack 與核心保存的 program counter、暫存器、排程狀態和資源。
- process 位址空間提供隔離。同一個 executable 被啟動兩次是兩個 process，各自有執行狀態。

五態：等待事件完成先回Ready，再由dispatch取得CPU



箭頭：合法state transition。單CPU running至多1個；mode切換本身不表示process離開running。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.1 Program不是process，PCB不是整個位址空間

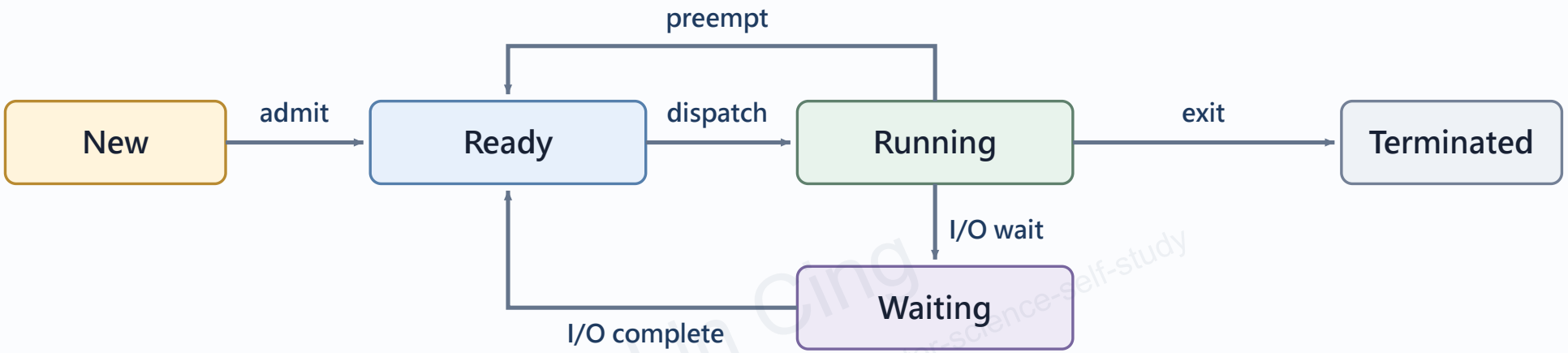
Program是可執行的指令/資料描述；process是一次具有身分、資源與執行狀態的執行。固定同一counter image啟動P100、P101，兩者可共享唯讀text的physical pages，卻各有private globals/heap/stack、register context與descriptor table。相同virtual address或fd number不等於相同physical object。

層次	P100	P101	共享的可能性
PID/PC/registers	PID100,PC0x40,r1=7	PID101,PC0x120,r1=3	每條執行流獨立
private counter@VA0x5000	0→1	0→1	無shared mapping時互不相加
fd3	description OA→file A	description OB→file B	編號相同但對象不同
code	相同image的read-only text	相同text	可共用frame，仍不同執行流

PCB保存核心需追蹤的身分、state、PC/registers、scheduling與memory/I/O references等；它可引用page tables和open descriptions，並不表示整個heap都直接塞進PCB。多threads時execution context會再分開，下一章詳述。

3.1　五態模型的轉移都有原因

五態：等待事件完成先回Ready，再由dispatch取得CPU



箭頭：合法state transition。單CPU running至多1個；mode切換本身不表示process離開running。

本章先用單CPU、每process單執行流模型：running至多1個；ready表示可跑但未分到CPU；waiting表示尚缺某事件，不能直接被當作ready候選。模型不含每個實際OS全部細分狀態，也不把terminated一律當作metadata已完全消失。

3.1 五態模型的轉移都有原因 (續)

Admission:new→ready；dispatch:ready→running；preemption:running→ready；blocking I/O/event:running→waiting；event完成:waiting→ready；exit:running→terminated。I/O完成通常只使process變ready，是否立即running由policy決定。等待者由別的CPU/interrupt喚醒，不需要「先自己跑一下才變ready」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.1 Context switch實際保存什麼

固定P100正在執行PC0x40、r1=7、SP0x9000，P101先前保存PC0x120、r1=3、SP0xA000。Interrupt/entry先保存架構要求的現場，核心把後續恢復所需context與task關聯，再載入P101的狀態；恢復後CPU不是沿P100的PC繼續。



3.1 Context switch實際保存什麼 (續)

時刻	CPU所屬	保存/載入結果
入口前	P100	PC40,r1=7,SP9000
保存P100	kernel	PCB100保留可恢復的40/7/9000
選中P101後載入	kernel→P101	取得120/3/A000及正確memory context
P100稍後再被選	P100	恢復自己的context，而不是P101的registers

模式切換不必換process；同process syscall進出就可只有mode transition。同process的兩threads切換可換register/stack卻不換user address space。實際PC保存點依ISA的interrupt/fault語意，不能假設所有事件都重跑同一條指令。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

3.2 Process scheduling

- Ready queue只包含可執行者，wait queue等待predicate；scheduler選人，dispatcher保存/載入context。
- Sleep、waiter登記與completion必須由同一同步協議銜接，否則會遺失喚醒；mode/task/address-space switch分開計。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.2 Queues、scheduler與dispatcher的分工

Ready queue是可執行集合；device/event wait queues是等待集合；job/admission管理決定哪些工作進入受資源約束的集合。Short-term scheduler選下一個ready執行流，dispatcher完成context/mode交接；long-term admission調整multiprogramming，medium-term suspension可暫時移出某些工作。實際OS未必都有獨立同名元件。

固定P1 running、P2 ready、R1磁碟尚未完成。P1送R1並阻塞→P2 running；R1完成後P1 ready；nonpreemptive policy允許P2繼續。Ready與waiting queues不得因重複completion使同一task被重複插入；running也不是額外再留一份可被別CPU同時dispatch的ready entry。

控制頻率會放大成本。固定每輪useful CPU10ms、scheduler/dispatch1ms，overhead fraction=1/(10+1)=1/11；若誤用1/10算的是相對useful work的額外比例，分母不同。這是固定成本模型，cache/TLB污染等實機成本需另量測。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

3.2 Sleep與completion競賽：一條喚醒不可遺失

固定done=false，process與completion handler共用lock L；等待者在L內檢done。若false，加入wait queue並設定WAITING，再釋L；之後只在仍WAITING時讓scheduler睡眠。Handler在L內設done=true，若有WAITING waiter則移出並設READY。這個協議把條件判斷、登記與狀態連起來。

completion時點	等待方觀察/狀態	結果
等待者取得L之前	看到done=true	不登記/不睡
等待者持L時發生	handler等L；waiter已登記才釋L	handler能找到它並改READY
登記後、真正schedule之前	done=true且state改READY	scheduler不可再把已喚醒者無條件睡下
已WAITING之後	handler正常喚醒	READY等待dispatch

錯誤寫法先check false、釋放保護、completion設true卻找不到waiter，然後waiter才登記並sleep；若無後續completion會永久等待。單靠「先登記waiter」幾個字不足以證明安全，還要指定同一lock、狀態轉移與schedule前重新確認。

3.2 CPU-bound與I/O-bound是資源需求，不是身分標籤

固定兩個工作：A交替CPU1ms/I/O9ms，B只做CPU10ms；單CPU與獨立I/O可重疊，A等待期間B可跑。若把所有工作都因當下CPU低使用率而再加入，可能使I/O queue或記憶體先超載。Admission需看完整資源需求與queueing，不能只看CPU平均值。

增大ready集合也不保證throughput單調上升；若working sets超出frames而thrashing，更多process反而更多page faults。暫停/喚回process屬medium-term資源控制，與每次CPU selection不同。具體排程演算法及waiting/response/turnaround的定義留第6章用共同workload比較。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.3 Operations on processes

- parent 可建立 child 並選擇共享或複製資源。終止 parent 時，系統可能終止子程序，也可能重新安置。
- zombie 已終止但尚未被 parent 收取狀態，orphan 的 parent 已不存在。收割與重新安置是資源生命週期的一部分。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.3 fork、exec、exit、wait是四個不同操作

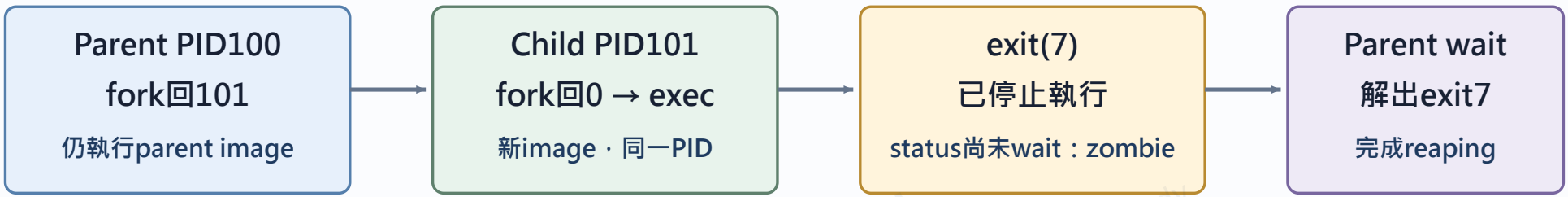
一般fork建立child process，child取得自己的task/address-space描述與必要資源引用；private pages可COW延後複製。exec用新program image替換呼叫者的image，成功不回原呼叫點；不額外建立一個process。exit結束執行並釋放大部分資源，wait使parent取得child狀態並完成相應reaping。

固定parent PID100、child101。fork在parent回101、child回0；child exec worker成功後仍PID101；worker exit7，在parent wait前保留可收取的exit status；wait辨識正常退出並解出7。Zombie是已終止、狀態尚未被收取；orphan是parent關係失效，child可仍在執行。重新指定監護者與收割規則依OS/namespace/subreaper政策，不能把兩個詞互換。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.3 fork、exec、exit、wait是四個不同操作 (續)

PID101：fork建立、exec換image、exit留status、wait收取



Parent先退出而child仍活著：orphan關係，不是上述已退出的zombie狀態。

箭頭：操作/生命週期順序。fork/exec均假設成功；不是four processes。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.3　兩個無條件fork為何得到4個process

固定每次fork都成功、無提前exit、只有原程式依序執行兩行fork，且兩行間沒有條件分支。第一次P0生P1；第二行由P0生P2，也由P1生P3，因此最後4個process，creation edges為P0→P1、P0→P2、P1→P3。

完成到哪一行	執行這一行的process	新child	存活總數
初始	P0	無	1
fork1	P0	P1	2
fork2	P0,P1	P2,P3	4

一般k次且每個現有process都到達下一行，N(k+1)=2N(k)、N0=1，歸納得2^k。若僅parent分支繼續fork、child立即exec/exit，或fork失敗，前提不成立；例如只有原parent再fork一次總數3。PID配置順序與print先後也不由這棵樹唯一決定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.3 繼承descriptors，不等於兩次獨立open

固定parent fd3引用O1，O1.offset=0。fork後child有自己的fd table entry，卻可指到同一O1；parent read100成功後O1.offset=100，child接著read50會從100起讀並變150。若另行open同一inode取得O2，O2.offset可仍0。這個共享關係與private user-memory COW分開。

fork失敗須走錯誤路徑而非當作child；exec失敗通常會回錯誤供child處理，但跨過特定實作的point of no return後不能無條件保證舊image完整保留。若parent不斷建立child而從不wait，退出狀態可累積。釋放一般記憶體/handles與留下最少reap資訊是不同生命週期。

Multithread process的fork在child只保留呼叫thread；其餘threads的mutex狀態可能被複製卻沒有對應持有者。第4章會限制child到exec之間可用的操作，不能把單thread流程直接搬用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.4 Interprocess communication

- shared memory 在建立後可高速交換資料，但 process 必須自行同步；message passing 把同步留給 kernel 或 channel 協定。
- pipe 有 capacity、阻塞與 EOF 語意。read、write 的回傳長度可能小於請求量，呼叫端不能只看一次結果。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.4 Shared memory與message passing的責任不同

Shared memory先建立共同backing/mapping，再由程序以loads/stores交換資料；不表示沒有權限檢查或自動同步。Message passing由send/receive契約決定copy、等待、容量與投遞；不表示每次必然跨machine，或零copy/固定兩次copy。

契約面向	必須明定的選項	混用的後果
Naming	直接process/endpoint，或mailbox	PID重用/權限/cleanup問題
Synchronization	send/receive各自blocking或nonblocking	blocking send可能僅等入列，不等對方處理
Capacity	零容量rendezvous、有界、理想無界	有界backpressure；無界仍有實體資源上限
Data boundary	完整message或byte stream	一次read不一定是一筆application record
Lifetime/failure	close、unlink、remove、peer crash	EOF、錯誤、queue消失不是相同事件

本章抽象channel capacity2保存兩個完整訊息。send J1/J2成功，send J3在full時等待；receive J1空出slot，才可使J3入列。若send只保證入queue，兩端都用blocking APIs也不必然是零容量rendezvous；須查完成條件。

3.4　SPSC ring：三個physical slots只容納兩個items

固定單producer、單consumer，共享slots[3]與atomic head/tail，初值0；head指下一個待取slot，tail指下一個待寫slot。保留一格辨別empty與full：empty為head=tail；full為next(tail)=head，next(x)=(x+1) mod3。有效items≤2，不是3。

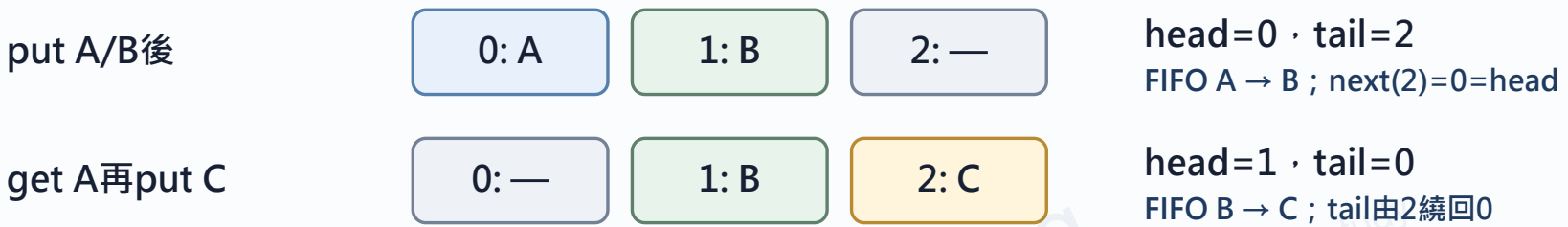
事件	head	tail	依FIFO尚未取出的items	結果
初始	0	0	[]	empty
put A至slot0	0	1	[A]	publish tail1
put B至slot1	0	2	[A,B]	full，next2=0=head
嘗試put C	0	2	[A,B]	不可覆寫slot2後公布
get A自slot0	1	2	[B]	release head1
put C至slot2	1	0	[B,C]	跨尾繞回仍full
get B，再get C	0	0	[]	empty

Lin Cing https://github.com/xinhi6716/computer-science-self-study

3.4　SPSC ring：三個physical slots只容納兩個items （續）

SPSC ring：3 slots保留1格；full時禁止覆寫

head=下一個待取，tail=下一個待寫；empty h=t，full next(t)=h。



write item → release tail；read item → release head

灰：邏輯空槽（舊bytes可仍在）。彩色：尚未消費items。箭頭/文字表示publish或reuse順序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.4 Publication與reuse的兩條happens-before

Producer acquire讀head，若非full，先寫slots[tail]內容，再release-store新tail。Consumer acquire讀tail，若非empty，先讀slots[head]內容，再release-store新head。Producer只寫tail、consumer只寫head；各自本地index操作與跨方load須符合明定的atomic規則。Process-shared atomics還需平台實際支援，不能把任意C++ atomic物件跨process映射就當作標準保證。

安全理由分兩條：producer的slot write先於release tail，consumer讀到相應publish之後的acquire使該slot資料可見，不能讀未完成item；consumer的slot read先於release head，producer在acquire看到slot被釋放後才可reuse，不能覆寫尚未讀完item。每次成功put/get使邏輯items加/減1，full/empty檢查維持0≤size≤2與FIFO序列。

若只用plain shared int，C/C++ data-race與memory-order問題使上述推理不成立；把tail提早公布則consumer可能看見舊slot。此演算法限SPSC，多producer會競爭同一tail/slot，不能只把threads數改大。有限trace驗證索引不變量，也不能代替release/acquire的一般同步論證。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

3.4 Byte-stream的framing必須自己組出完整record

固定每則record為2-byte big-endian長度加payload，最大payload8bytes。兩則payload為ABC、DE，stream bytes為00 03 41 42 43 00 02 44 45，共9bytes。read可分成[00]、[03 41 42]、[43 00 02 44 45]，不能把這三次返回當三則message。

Parser先收齊2-byte header得到3，再收齊ABC；下一個header得到2，再收齊DE。EOF若落在半個header或不足payload時是truncation，不是正常完成一筆空訊息。先限制length≤8與檢查配置算術，再收payload，避免對不可信長度盲目配置。

TCP與pipe都可提供ordered byte-stream情境，但可靠連線不替應用提供record邊界，也不讓remote procedure自動atomic。Datagram/message API的邊界與截斷規則則要按各自契約處理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.5 IPC 系統例子

- POSIX shared memory的name/descriptor/mapping，POSIX MQ unlink與System V IPC_RMID的生命週期不同。
- Mach port rights與Windows教材ALPC channel各有命名/權利/reply契約；STOP/drain是application protocol，不能冒充原生remove保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.5　POSIX shared memory：名稱、descriptor、mapping分開

固定creator建立`/course-ring`，設4096-byte長度後MAP_SHARED映射；consumer開同名物件再map。兩個virtual base可不同，例如P1@0x100000、P2@0x700000，都可看到backing同一offset16。跨process資料應用offset/index而非直接存P1的absolute pointer給P2。

關descriptor不自動解除既有mapping；shm_unlink移除名稱，後續open不再藉舊名稱找到原物件，但既有引用/mappings可繼續有效直到相應生命週期結束。MAP_SHARED只提供共同backing，不自動保證先初始化完再開始讀；需ready protocol與process-shared同步。若consumer在初始化length前就觸碰超出物件有效範圍，不能當作普通zero-filled buffer。

Shared region中先放magic/version/length與offsets，producer完成初始化後才用受同步保護的ready條件通知consumer。這是publication協議，不可用「sleep一秒應該夠」取代。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.5 System V remove與POSIX unlink不是同一種close

API物件	名稱/識別	移除動作	既有使用者
POSIX message queue	name與open descriptors	mq_unlink立即移除name	既有open引用可續用，到最後close後銷毀
System V message queue	msqid/key	IPC_RMID移除queue	blocked send/receive會被喚醒並回EIDRM
本章抽象drain channel	自訂handle	close-for-send，drain，最後destroy	必須按明定狀態機，不能叫它原生API保證

因此舊直覺「remove queue一定等consumer drain才成功」只可能是自訂application protocol，不能寫成System V或POSIX共同保證。解除name也不是自動送一個EOF給每個receiver；使用者要自行約定終止與錯誤處理。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

3.5　用明確sentinel結束一個consumer

固定FIFO application queue capacity4，單producer/單consumer，合法jobs只含正整數，0保留STOP。Producer送1、2、3後禁止新jobs，再送0。Consumer依次完成1/2/3，再取到0並ack停止，管理者收到ack後才做適當cleanup。Queue內容依序[1]→[1,2]→[1,2,3]→[1,2,3,0]；這是應用協議，不是kernel自帶drain承諾。

若job0合法就不能再用0作sentinel；改用有明確tag的message。若有2個consumers而只送1個STOP，另一個可能永遠等，須設計2個STOP或另有broadcast/closed state。若consumer crash，ack可能永遠不到，還要指定重派/恢復policy；不能把一個正常路徑當exactly-once job處理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.5　Mach：port rights、reply port與訊息順序

以本書Mach案例的port/mailbox抽象為準：server持有receive right，client須有相應send right；訊息header標destination與reply port，typed body可攜帶資料或受控rights。知道某個port名稱不等於有權receive，也不能把一個process內的local name當全系統通用pointer。

固定client A送A1/A2、client B送B1，各自send完成順序A1先A2。允許A1,B1,A2或B1,A1,A2，但不應讓同一sender的A2排在A1之前；跨sender沒有自動全域先後。Server處理A1後對其reply port回覆，A2/B1仍可能等待，send成功不等於reply成功。

若queue已滿，send可採wait、timeout或立即失敗等明定選項；接收方需有receive right才能取。Out-of-line memory可利用VM mapping/COW減少大資料複製，不能由此推論所有inline headers、所有跨host messages都zero-copy。傳送權利與共享memory的生命週期也需隨協議處理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.5 Windows教材ALPC：connection建立後才有private channel

以本書Windows案例的抽象，server公開connection port；client取得受權handle並送connection request；server接受後建立private communication channel與兩方向的communication endpoints，client/server用handle引用各自可用物件。Connection endpoint用於建立通道，不是把每個payload都重新當作新connection。

教材以小訊息queue copy與大資料shared section區分資料路徑，並使用256-byte作當時案例閾值；本章不將它當現代各版Windows固定API限制。128-byte request可走所述small-message模型，4096-byte payload可用預先建立的section加offset/length控制訊息。Section避免重複大資料copy，但mapping範圍、權限與誰可寫仍要驗證；不能信任client任意pointer。

應用通常經RPC等公開介面間接使用此local機制。Local RPC/ALPC、Mach port與POSIX queue的共同研究點是命名、權利、等待、資料路徑和回覆；其rights/close/queue語意不能僅換函式名稱互套。

3.6 Client-server 通訊

- socket 讓程序跨主機交換資料。peer 要能命名彼此，RPC 把遠端呼叫包成較熟悉的介面。
- 遠端呼叫可能重送、延遲或只完成一半。at-most-once、at-least-once 與 exactly-once 需要不同的持久紀錄與去重機制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.6 Socket endpoint與RPC stub各負責什麼

Socket以address family、local/remote address與port等資訊識別通訊端點；stream server通常bind/listen/accept，accept回新的connected endpoint，listening socket仍能接後續connections。Port不是process PID；同一server也可持多個connections。Client connect成功不證應用操作已完成。

RPC的client stub編碼procedure id、request id與parameters，server stub驗證/解碼後呼叫實際服務並編碼結果。不同ISA的endianness、alignment、integer widths與pointer值不能直接當network格式。

固定wire格式為request id32bit、opcode16bit、amount32bit，全big-endian。id77、opcode1、amount25編成00 00 00 4D | 00 01 | 00 00 00 19，共10bytes。Server先驗長度與opcode，再解amount；把client pointer地址送過去不會自動讓server可讀client memory。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

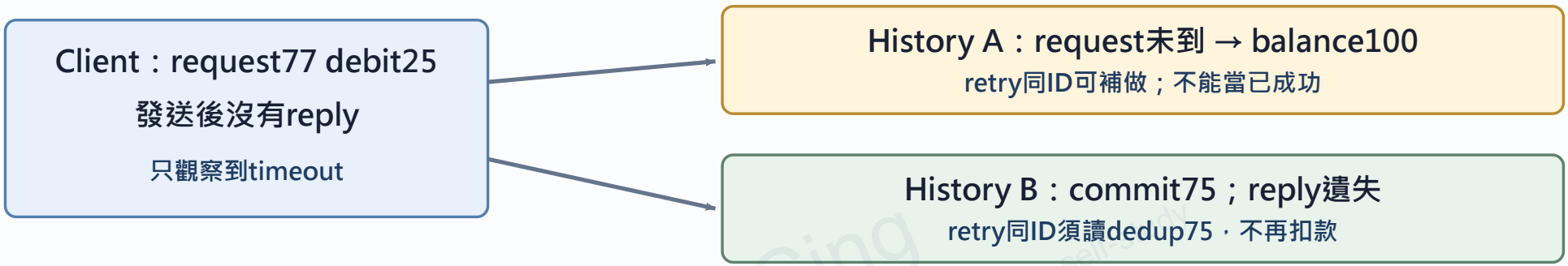
3.6　遺失reply後的timeout有兩個相同觀測history

初始balance100，request77要debit25。History A：request未到server，balance100；history B：server已commit balance75，但reply遺失。Client在兩者都只看到timeout，故不能僅憑timeout辨認是否執行。無條件換新ID重試在B會變50；不重試在A則遺漏工作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.6　遺失reply後的timeout有兩個相同觀測history （續）

相同client timeout，不能判定debit是否已commit



balance更新 + (client,id77,result75) 必須同一可恢復atomic commit；兩次分離write不足。

若需重試而不重複effect，server以(client identity,request id)查dedup；更新balance與結果紀錄必須在同一可恢復atomic transaction中commit。第一次77由100→75並記result75；重送77回75而再扣0次。並行兩個77也需transaction/unique constraint串行化，不能都先查「不存在」再各扣一次。

3.6　At-most-once與可用性是不同保證

保留dedup records期間可保證同IDeffect至多一次；若server在debit後、記id前crash，分離兩次write無法達成。若record只在RAM，重啟遺失後同ID可重做；若清掉舊id後client還能重試，也需要epoch/保留期限協議。Request id空間與authenticated client身分都不能任意重用。

At-most-once不保證一定執行：request永久丟失時可能零次。At-least-once假設重試與最終可達時提高完成機會，但non-idempotent操作可重複。網路下無條件exactly-once的口號不足以實作；本例只在transaction、持久紀錄、ID作用域、故障模型與終究通信等條件下討論effect與結果。純read或設值可能idempotent，debit25則不是。

Anonymous pipe與named FIFO解決同機stream命名/繼承，socket支援更一般的端點與跨host；三者仍需framing、backpressure與peer failure處理。IPC不是只有資料copy成本，還包含喚醒、同步、物件引用與重試。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.7　程序生命週期與資料生命週期接在一起

固定producer P與consumer C，channel capacity2。P送J1/J2後full，嘗試J3而等待；C取J1釋放一格，P回ready並在被選後送J3。Queue仍FIFO且不超過2；wakeup不表示producer當場執行。Consumer取到item不等於job effect已持久完成，結束程序也不代表parent已wait收取狀態。

不同層的身分需分開：PID與PCB描述執行；fd entry引用open description；shared-memory offset指資料；request id辨別重試effect。相同編號或同一pathname不能取代物件引用與生命週期證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.7　哪些條件支持這一章的安全推理

Context保存/恢復要對同一執行流；sleep與completion要連接同一受同步保護的predicate；ring先寫資料再publish、讀完才release供reuse；stream parser只輸出完整且合法length的records；RPC的effect與dedup result共用commit。

對每種模型，已給出的固定trace可重算中間狀態，一般不變量則說明更多容許事件為何仍安全。有限測試不證明所有interleavings或crash都正確；實作若更換API、增加producer/consumer、改記憶體順序或改持久策略，必須重新檢查相應前提。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料

- *Operating System Concepts*, 9th Edition, Chapter3，印刷頁105–147；§3.1 Fig3.4 context switch與§3.4 Fig3.13 bounded-buffer角色/索引；§3.5的Mach與Windows ALPC是教材歷史案例。
- pipe(7)：byte-stream與writer引用/EOF；本章自訂atomic message channel另明定契約。
- msgctl(2)、mq_unlink(3)：System V移除與POSIX unlink生命週期差異。
- pthread_atfork(3)：multithread fork只保留caller的背景與鎖問題；下一章給具體限制。
- SPSC的release/acquire與平台process-shared支援、RPC共同commit/dedup以及stream framing均明列為模型前提；範例程式模擬順序狀態；ring methods視為atomic操作，ledger狀態更新視為單一可恢復commit。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study